AsymptoticAnalysis: A Native-Boundary Audit

Abstract

The current AsymptoticAnalysis repository combines a specialized real asymptotic algebra with a newer adapter to Wolfram Language's native expansion engines. Its strongest distinction is not simply a longer list of expandable functions: it is the representation of ordered real-power and logarithmic blocks together with branch, remainder, and refinement information. The native adapter correctly avoids automatically promoting arbitrary native output into that analytic contract. However, the surrounding request and result machinery has independently reproducible defects.

This audit establishes four concrete defect families and one diagnostic-design deficiency: backend defaults changed by `SetOptions` are ignored; computed option keys are classified too early, causing dispatch and variable-metadata failures; an unrecognized symbolic option can be accepted without a failure result; native sparse results are eagerly converted into dense stored expressions; and a returned failure sentinel can be labelled `"Computed"`. Two narrow repair mechanisms were exercised independently in a native kernel. A separate documentation inconsistency is identified. Each finding is compared with the existing eighteen-review register, rather than treating earlier discoveries or completed fixes as new work. The accompanying artifacts include reproducible probes, desired-contract tests, a guarded patch emitter, an outcome-inspection prototype, and explicit execution boundaries.

Revision and evidence boundary

The subject is commit `6687962f3c858a4f93623cfc496f33e35c6763d4`, not a moving `main`. Its commit time is September 10, 2026, 00:18:17 UTC, or September 9, 2026, 17:18:17 PDT. The package is named `AsymptoticAnalysis`; `AsymptoticInverse` remains a public operation, not the current package context. The native observations in this article used Wolfram Language 15.0.0 for Linux x86-64, dated May 6, 2026. Upstream Windows 15.0.1 acceptance records describe their own named snapshots and are not substituted for these observations.[1,2]

The review combines the repository's source map, public declarations, maintained review register and wave indexes, close inspection of the native dispatcher and selected mathematical boundaries, current official Wolfram documentation, and focused native execution. It is not a line-by-line certification of every module or a rerun of the full upstream suite. Unsuccessful service/network calls were not classified as package failures. The two source replacements described below were tested separately; a complete patched-package acceptance run is not claimed.

[1] AsymptoticAnalysis, pinned revision `6687962f3c85`, `README.md`, package identity, public interfaces, and validation links..

[2] Accompanying `evidence/native_observations.json`, observation(s) O01–O13; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

Contents

1 Assessment and priorities

1.1 What the package adds

AsymptoticAnalysis is best understood as a contract-bearing real asymptotic layer that can also carry native results without claiming the same contract for them. The maintained interface includes real power-logarithmic forward expansions and inverse expansions, finite and infinite endpoint charts, selected logarithmic and exponential cores, selected flat and oscillatory scales, arithmetic on expansion objects, refinement, residual diagnostics, and a restricted exact-rational inverse-certificate facility. The specialized Gamma and Barnes inverse families preserve structured cores and use complete correction blocks rather than reducing everything to an ordinary Taylor polynomial. These are meaningful features, not merely aliases for `Series`.[3]

Conversely, native Wolfram functionality is substantially broader than ordinary integer-power Taylor expansion. `Series` supports Laurent and fractional-power behavior, logarithmic terms, infinity, and successive-variable expansions. `Asymptotic` can produce expansions in non-Taylor scales. Therefore, a comparison claiming that Wolfram's built-ins cannot handle logarithms, exponential asymptotics, or inverse-series problems would be misleading. The useful comparison is between admitted inputs, supported representations, operation closure, and retained evidence.[4,5]

1.2 Finding summary

The priorities below are engineering judgments, not security ratings. In particular, none of the new findings is presented as a newly demonstrated false root certificate or a newly demonstrated wrong irrational inverse coefficient.

ID	Finding	Classification	Recommended action
N01	Configured backend default is ignored	Confirmed; medium	Resolve the missing selector from the canonical operation's option defaults.
N02	Computed keys break routing and variable metadata	Confirmed; medium	Resolve admitted key expressions before assigning argument roles, preserving held source evaluation.
N03	Unknown symbolic option receives a successful package result	Confirmed validation gap; medium	Use position-aware specification parsing and reject unconsumed option rules.
N04	Native sparse output is densified during wrapper construction	Confirmed resource defect; high for large arrays	Preserve the native payload and materialize `Normal` only on explicit demand.

[3] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/AsymptoticAnalysis.wl`, public usage declarations, lines 14–116..

[4] Wolfram Research, *Series*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/Series.html`.

[5] Wolfram Research, *Asymptotic*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/Asymptotic.html`.

ID	Finding	Classification	Recommended action
N05	Failure values satisfy the `"Computed"` classifier	Confirmed behavior; diagnostic-design issue	Separate syntactic resolution from returned failure/partial-failure status.
D01-local	README describes an older automatic-routing policy	Documentation defect; low	Replace the stale paragraph and test its executable policy examples.

N02 and N03 have a shared contributor: an overly permissive specification classifier. They are separated because one rejects or damages a valid request, while the other accepts an invalid or ambiguous trailing argument. This is not a claim of five completely unrelated root causes. N05 behaves according to its documented syntactic rule; the criticism is that this rule is too weak to serve as a reliable outcome signal.

N01 and N04 deserve early attention. The former defeats a policy that may be set once for an entire notebook or application. The latter changes the storage complexity of a supported native request before the caller asks for the expensive representation. N02 has a small candidate repair with successful focused evidence, while N03 requires an explicit grammar decision rather than another isolated pattern exception.

2 Novelty relative to the existing reviews

2.1 The comparison baseline

The review directory contains eighteen packages in two waves. Wave 1 covers older `07a9781/75de875` snapshots; wave 2 covers `921387e`. The maintained implementation register consolidates 123 identified entries and distinguishes completed focused repairs, pending work, audit candidates, and deferred extensions. The present novelty screen uses that register and both wave indexes, with the current source as the authority for whether a problem still applies. It does not claim that all eighteen articles were independently re-proved or that every supplied historical patch was executed.[6,7,89]

The register already covers major algorithmic and mathematical themes: remainder transport, refinement precision, exact exponent grouping, parameter dependence in composition, resource accounting, proof evidence, and larger transseries extensions. Those themes are not repackaged here as discoveries. Nor are corrected assumption capture, corrected pure-uncertainty power guards, or the old optional `SeriesData` allocation problem reported as current new bugs.

2.2 Finding-by-finding crosswalk

[6] AsymptoticAnalysis, pinned revision `6687962f3c85`, `external-reports/code-review/README.md`, review inventory and evidence policy..

[7] AsymptoticAnalysis, pinned revision `6687962f3c85`, `external-reports/code-review/wave-1/README.md`, review subjects and original snapshot boundaries..

[8] AsymptoticAnalysis, pinned revision `6687962f3c85`, `external-reports/code-review/wave-2/README.md`, incremental review subjects and native-execution limits..

[9] AsymptoticAnalysis, pinned revision `6687962f3c85`, `docs/development/CODE_REVIEW_STATUS.md`, the named register entries and complete finding crosswalk..

ID	Nearest earlier items	Incremental contribution
N01	C09, D01, B02	C09 concerns precedence of a stored inverse-coefficient power. The new witness is a changed published backend default that is never read by dispatch; it changes the selected representation and order convention.
N02	D06, B02–B03	General interface compatibility is already a goal. The new counterexamples identify a specific evaluation-stage error and a false additional variable created from a native option alias.
N03	D01–D02, B02	This is an actual accepted unknown-symbol rule after a complete expansion specification, not merely a proposal to improve diagnostics.
N04	C01, P06–P08, B03	C01 concerns an optional dense `SeriesData` view of an analytic sparse expansion. N04 starts with a native `SparseArray` that the native engine already preserved; the new wrapper then densifies it unconditionally. The concrete family and measured storage distinguish the sites.
N05	D02, B03	General result-contract discussions are extended by explicit returned `$Failed` and `Failure` values classified as computed. The proposed flags preserve the published syntactic observation while supplying the missing outcome information.
D01-local	B01–B02	The current README's development-status paragraph contradicts both its own introduction and the implemented automatic dispatcher. This does not imply automatic fallback is absent.

The supplied `evidence/novelty_ledger.csv` makes these relationships machine-readable. Existing register IDs retain their upstream meanings; `D01-local` is intentionally qualified to avoid confusing it with the register's existing D01.

3 Architecture and mathematical contracts

3.1 The important architectural boundary

The canonical modular entry point declares public operations and shares a private implementation context with its companion modules. The generated repository-root standalone is a distribution form, not a second independent implementation. The source map identifies separate areas for coordinate charts, inverse cores, native compatibility, special-function admission, arithmetic, refinement, and certificates. This organization is useful, but a module boundary alone does not establish a semantic boundary: the dispatcher and the constructors determine which kind of result is actually produced.[10]

A useful conceptual decomposition is

$$\begin{aligned} &\text{request} \longrightarrow \text{resolved policy and argument roles} \longrightarrow \text{engine},\\ &\text{engine} \longrightarrow \text{result contract} \longrightarrow \text{optional views and operations}. \end{aligned}$$

[10] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/README.md`, source map and load-order policy..

The newly confirmed defects occur in the first and last transitions, not in the coefficient recurrence used for the irrational inverse example below. That distinction matters for repair strategy. It would be disproportionate to rewrite the asymptotic algebra to fix a selector that ignores `Options`, or to redesign the inverse enumerator to fix eager `Normal` on a sparse native result.

3.2 Ordinary real power-logarithmic results

The declared ordinary model has the form

$$f(x) = y_0 + au^p \left(1 + \sum_i u^{\delta_i} B_i(\log u)\right), \qquad u \to 0^+,$$

where the coordinate u describes the selected source endpoint and side. Finite endpoints use a positive local displacement; infinite endpoints use a reciprocal magnitude coordinate. The coefficient blocks are polynomials in a logarithm, while powers need not lie on a single rational lattice. The remainder descriptor

$$\text{PowerLogRemainder}(u, \beta, k) \quad \text{denotes} \quad O\Big(u^\beta (1 + |\log u|)^k\Big).$$

These meanings are part of the public declarations, not inferred from a pretty printed expression.[11]

For a fixed analytic model, an exclusive power cutoff is a clear operation: retain complete blocks with exponent strictly smaller than the cutoff. A request for a number of nonzero blocks is different, and a specialized extracted prefactor may change the coordinate in which the cutoff is interpreted. The repository already documents these distinctions. A wrapper that accidentally selects a different engine can therefore change more than output formatting: it can change what the request means.

The result objects expose finite expressions, terms, remainders, branch-related information, and operation/refinement state. Removing the wrapper with `Normal` discards information needed by later analytic operations. A retained symbolic O-class also does not specify a numerical constant or a certified pointwise error. That is why the package's separate residual, numerical-check, and exact-rational certificate interfaces should remain separate.[12]

3.3 A checked irrational-support inverse

Consider the positive real branch of

$$y = x + x^\alpha, \qquad \alpha = \sqrt{2}, \qquad x \to 0^+.$$

The derivative $1 + \alpha x^{\alpha-1}$ is positive on $x > 0$, so this branch is locally single-valued and increasing. Set $t = y^{\alpha-1}$ and $x = y(1 + w)$. Then

$$w = -t(1 + w)^\alpha.$$

[11] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/AsymptoticAnalysis.wl`, `PowerLogModel`, `PowerLogRemainder`, and constructor usage declarations..

[12] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/AsymptoticAnalysis.wl`, `GeneralizedSeries`, `InverseResidual`, `InverseNumericalCheck`, and `InverseCertificate` usage declarations..

Around $(w,t)=(0,0)$ the derivative of $w+t(1+w)^\alpha$ with respect to w is one. Consequently w has an ordinary convergent local expansion in t. Substituting $w=c_1t+c_2t^2+c_3t^3+O(t^4)$ and comparing powers gives

$$c_1=-1, \qquad c_2=\alpha, \qquad c_3=-\frac{\alpha(3\alpha-1)}{2}.$$

Thus

$$x=y-y^\alpha+\alpha y^{2\alpha-1}-\frac{\alpha(3\alpha-1)}{2}y^{3\alpha-2}+O(y^{4\alpha-3}).$$

In particular, the expansion retaining exponents below two is

$$x=y-y^{\sqrt{2}}+\sqrt{2}\,y^{2\sqrt{2}-1}+O(y^{3\sqrt{2}-2}),$$

because $2\sqrt{2}-1<2<3\sqrt{2}-2$. This derivation supplies an independent mathematical check of both the retained coefficients and the reported next frontier.

The following current package call was executed successfully:

```
s = AsymptoticInverse[x + x^Sqrt[2], {x, 0}, {y, 2}];
{Normal[s], s["Remainder"]}
(* {y - y^Sqrt[2] + Sqrt[2] y^(-1 + 2 Sqrt[2]),
    PowerLogRemainder[y, -2 + 3 Sqrt[2], 0]} *)
```

The native comparison is instructive but should not be exaggerated. On the tested kernel, `Series[x+x^Sqrt[2],{x,0,3}]` returned an irrational term added to a `SeriesData` object; direct `InverseSeries` of that sum remained unevaluated. The native composition does not provide the same structured inverse result on this witness. That observation does not establish that every possible native reformulation or `AsymptoticSolve` route must fail.[13]

3.4 Specialized domains and certification

The inspected special-function real-domain code uses sufficient real-domain conditions and explicitly distinguishes real-valuedness from an asymptotic-validity or continuity certificate. It checks parameter reality before relying on real quantifier reasoning and applies endpoint-specific restrictions to functions with cuts or poles. This is a useful conservative boundary. It must not be mistaken for a proof that every accepted formal Taylor expansion has the analytic remainder needed by the custom algebra; the existing register already tracks that separate admission question.[14][15]

Similarly, the elementary exponential-forward adapter separates a multiplicative coefficient from a logarithmic phase, checks an eventual coefficient sign and real phase conditions, and reserves its extracted exponential treatment for growth beyond a merely logarithmic source. These checks

[13] Accompanying `evidence/native_observations.json`, observation(s) O12–O13; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

[14] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/SpecialFunctionRealDomain.wl`, introductory contract and sufficient-domain walker..

[15] AsymptoticAnalysis, pinned revision `6687962f3c85`, `docs/development/CODE_REVIEW_STATUS.md`, the named register entries and complete finding crosswalk..

preserve the distinction between ordinary power-law factors and genuinely exponential scales. No new failure of that mechanism was established here.[16]

The inverse-certificate implementation inspected here uses exact rational endpoints, directed dyadic rounding, elementary-function enclosures, domain checks, derivative information, and a separate accuracy/refinement loop. Its recent relative-accuracy changes are already recorded under C19; they are not rediscovered in this article. The relevant conclusion for the new dispatcher work is narrower: choosing a native representation must never manufacture access to this analytic certificate contract, and respecting a configured strict-package policy is therefore a meaningful API guarantee.[17]

4 Comparison with built-in Wolfram Language

4.1 Capabilities and representations

Task	Native Wolfram facilities	AsymptoticAnalysis position
Local expansion	`Series`, `SeriesData`, `Normal`; Laurent, fractional powers, logarithms and successive specifications are not absent from the native system.	Custom ordered real-power/logarithmic blocks and explicit analytic remainder descriptors on admitted inputs; otherwise selected native delegation.
Series reversion	`InverseSeries` reverts appropriate `SeriesData`; `AsymptoticSolve` addresses asymptotic equation solving.	Direct real-branch inverse interface, irrational-support example above, selected exact-core inverse families and retained inverse state.
Non-Taylor asymptotics	`Asymptotic` handles inferred asymptotic scales and special-function examples.	Specialized scale models can retain correction-block structure and admitted operation contracts; native backends extend input reach without conferring those contracts.
Arithmetic and composition	Native series arithmetic and `ComposeSeries` work within their native representations.	Custom operations transport the package's own remainders where supported; Native results are explicitly excluded from analytic operations unless independently promoted.
Integration and summation	`AsymptoticIntegrate` and `AsymptoticSum` are dedicated native tools.	Not a general replacement for those solvers. Earlier adapter and integration proposals are already in the register.
Differential and recurrence equations	`AsymptoticDSolveValue` and `AsymptoticRSolveValue` supply specialized native interfaces.	A useful result/contract layer is a different objective from implementing an entire ODE or recurrence solver.

[16] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/ExponentialForward.wl`, `logarithmicProductData`, `logarithmicProductSource`, and `exponentialForwardExpansion`..

[17] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/InverseCertificates.wl`, rational interval helpers, lines 1–180, and certificate control flow, lines 300 onward..

Task	Native Wolfram facilities	AsymptoticAnalysis position
Numerical root evidence	Numerical solving and native symbolic mathematics can support checks, but are not the same as this package's explicit rational certificate record.	Separates an expansion, a finite residual, numerical comparison, and a restricted original-equation certificate.
Structured arrays	Native output can retain a `SparseArray`, as directly observed.	`NativeResult` retains it, but eager `Expression=Normal[result]` introduces the new N04 storage defect.

The native facilities in this comparison are documented separately; their existence is not inferred from package marketing.[18,19,20,21,22,23,24]

4.2 Observed controls, rather than blanket superiority claims

Several small current-kernel observations anchor the comparison. Native reversion of the regular series for $x + x^2$ yields $y - y^2 + 2y^3$ at the tested order. Native `Asymptotic[Erfc[x],{x,Infinity,3}]` returns the first displayed e^{-x^2}-scaled inverse-power terms in this call. Native `Asymptotic[Gamma[x],{x,Infinity,3}]` likewise produces Stirling-scaled terms. Therefore neither exponential prefactors nor basic special-function asymptotics constitute an exclusive package capability.[25]

There is deliberately no package-versus-native speed league table. A number such as "order three" is not a common unit across the APIs, and the tests were not a controlled throughput benchmark. Matching calls must first match the retained information: the cutoff coordinate, inclusivity, complete-block convention, endpoint, side, assumptions, and whether the requested result is formal or analytic. A faster call producing a different approximation is not a valid comparison.

The package's strongest demonstrated distinction in these controls is the explicit irrational inverse with a next-frontier remainder. The native wrapper's strongest design decision is the refusal to reinterpret arbitrary native output as a real analytic theorem. Its weakest newly exposed point is that ordinary Wolfram request and representation behavior is not yet preserved consistently around the selected engine.

[18] Wolfram Research, *InverseSeries*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/InverseSeries.html`.

[19] Wolfram Research, *AsymptoticSolve*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/AsymptoticSolve.html`.

[20] Wolfram Research, *ComposeSeries*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/ComposeSeries.html`.

[21] Wolfram Research, *AsymptoticIntegrate*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/AsymptoticIntegrate.html`.

[22] Wolfram Research, *AsymptoticSum*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/AsymptoticSum.html`.

[23] Wolfram Research, *AsymptoticDSolveValue*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/AsymptoticDSolveValue.html`.

[24] Wolfram Research, *AsymptoticRSolveValue*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/AsymptoticRSolveValue.html`.

[25] Accompanying `evidence/native_observations.json`, observation(s) O12; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

4.3 Order semantics are part of correctness

For e^x near zero, the package's exclusive cutoff three and native `Series` order three produce, respectively,

$$P_{\text{package}}(x) = 1 + x + \frac{x^2}{2}, \qquad P_{\text{native}}(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{6}.$$

Their difference is $x^3/6$. Both are legitimate approximations under their own conventions. Returning the former after the caller explicitly configures the latter backend is nevertheless a request-semantics defect. The distinction is not cured by observing that both converge to e^x as $x \to 0$.

A comparison framework should therefore keep the engine identity and order convention alongside coefficient equivalence. Matching only `Normal` after truncating both outputs to a common lower order would hide N01. Matching only the native payload would hide N04's unwanted extra dense materialization. These are examples of why a coefficient-only test matrix is incomplete.

5 N01: backend defaults are advertised but not consulted

5.1 Reproduction and impact

After loading the pinned standalone in a fresh kernel, execute:

```
SetOptions[AsymptoticExpansion, "Backend" -> "Series"];
s = AsymptoticExpansion[Exp[x], {x, 0, 3}];
{Options[AsymptoticExpansion, "Backend"], s["Kind"], Normal[s]}
(* {{"Backend" -> "Series"}, "Forward", 1 + x + x^2/2} *)
```

The declared default changes successfully, but the constructor takes the package route and omits the cubic term native order three would retain. The converse policy example is more consequential:

```
SetOptions[AsymptoticExpansion, "Backend" -> "Package"];
s = AsymptoticExpansion[Exp[I x], {x, 0, 3}];
{s["Kind"], s["NativeBackend"]}
(* {"Native", "Series"} *)
```

An explicit per-call `"Backend"->"Package"` control refuses the same complex input with tag `"InexactInput"`. The tag's wording is not the new finding; the significant observation is the disagreement between configured and explicit policy. The returned Native result still carries missing analytic contracts, so this example is not a false analytic certificate.[26]

`SetOptions` is documented as changing a symbol's default options. For a function that exposes `"Backend"` in `Options`, ignoring the changed default violates the ordinary expectation of this interface. A client that sets the backend once at initialization has no reason to know that this particular option is bypassed by the wrapper's custom dispatcher.[27]

[26] Accompanying `evidence/native_observations.json`, observation(s) O02–O03; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

[27] Wolfram Research, *SetOptions*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/SetOptions.html`.

5.2 Cause and minimal repair

The dispatcher performs literal selector discovery and then uses

```
backend = If[values === {}, Automatic, ReleaseHold[First[values]]];
```

The missing-selector branch is a constant, not an option lookup. Changing `Options[AsymptoticExpansion]` therefore has no effect on this branch. The later selected engine receives a valid request, so engine-specific coefficient tests need not expose the problem.[28]

A narrowly scoped replacement is

```
backend = If[values === {},
  OptionValue[AsymptoticExpansion, {}, "Backend"],
  ReleaseHold[First[values]]];
```

The three-argument `OptionValue` form reads the explicit option list and the named function's defaults. Here the explicit list is empty because literal explicit backend values have already been selected by the dispatcher. This preserves explicit-value precedence while making the absent-value path honor the canonical default.[29]

This replacement was applied to a temporary in-memory copy of the pinned standalone, after verifying that the original anchor occurred exactly once. The repaired O02 witness returned `"Native"`, `"Series"`, and $1 + x + x^2/2 + x^3/6$. This establishes the selected mechanism, not full acceptance of every delayed-default or nested-option case.[30]

5.3 Remaining acceptance obligations

The default repair should be tested for each backend value, an explicit override of a changed default, a delayed default with a counting callback, nested selectors, and the held alias. There is a separate alias-policy choice: does `AsymptoticExpand` have its own mutable defaults, or is it a true alias whose canonical defaults belong to `AsymptoticExpansion`? Its current initial `Options` copy and forwarding definition do not by themselves define a coherent policy for subsequent mutations. This is a source-level design question; no separate alias-default native counterexample is claimed here.

The smallest stable policy is one canonical option owner, with the alias forwarding to it and documentation making that ownership explicit. A broader alternative is independently configurable public entry points that resolve their own defaults before invoking one internal request constructor. Either policy is defensible. Leaving a public option list that has no operational effect is not.

[28] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/NativeCompatibility.wl`, `expansionDispatch`, especially lines 65–80..

[29] Wolfram Research, *OptionValue*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/OptionValue.html`.

[30] Accompanying `evidence/native_observations.json`, observation(s) P01; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

6 N02: computed keys are classified before they are resolved

6.1 A valid selector is missed

A held front end should not evaluate arbitrary source programs merely to inspect their options. The current dispatcher correctly attempts to preserve that boundary. The problem is more specific: it declares every literal rule or delayed rule to be already prepared, even when the rule's key is a computed expression.

This witness fails on the pinned source:

```
Module[{key = "Backend"},
  AsymptoticExpansion[Exp[x], {x, 0, 3}, key -> "Series"]]
(* Failure["NativeOptionConflict", ...
     "Options" -> {HoldComplete["Backend"]}] *)
```

The same requested backend written with the literal string is accepted. In the failing case the key is eventually evaluated to `"Backend"`, but only after the first selector-discovery pass has missed it. The now-materialized selector is subsequently treated as an incompatible native option rather than as the wrapper's backend choice.[31]

6.2 An option alias becomes a false expansion variable

A second witness has an externally visible consequence beyond routing:

```
opt = Analytic;
s = AsymptoticExpansion[Exp[x], {x, 0, 3},
  opt -> False, "Backend" -> "Series"];
{s["Variable"], s["ExpansionSpecifications"], s["Expression"]}
(* {Missing["MultipleOrUnresolvedVariables"],
     {HoldComplete[{x, 0, 3}], HoldComplete[opt -> False]},
     1 + x + x^2/2 + x^3/6} *)
s[1]
(* Failure["NativeVariables", ...] *)
```

The native engine produces the correct univariate expansion, but the wrapper records the aliased option as another expansion specification. Numerical application then refuses because it believes there are multiple or unresolved variables. This is not merely a less attractive metadata display; a documented public operation becomes unavailable on an otherwise valid univariate result.[32]

6.3 Root cause

Two rules interact:

[31] Accompanying `evidence/native_observations.json`, observation(s) O04; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

[32] Accompanying `evidence/native_observations.json`, observation(s) O05–O06; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

```
nativeComputedArgumentQ[HoldComplete[_Rule | _RuleDelayed]] := False;

nativeSpecificationQ[
  HoldComplete[(Rule | RuleDelayed)[key_Symbol, _]]] :=
  ! MemberQ[First /@ Options[AsymptoticExpansion], Unevaluated[key]];
```

The first suppresses preparation of a computed key. The second treats an unrecognized symbolic key as a variable specification. When `opt` has the value `Analytic`, it is still syntactically `opt` at this classification boundary. Native evaluation later recognizes the option correctly, but the wrapper's role assignment is already inconsistent with the effective call.[33]

The result-contract document deliberately describes `ExpansionSpecifications` as syntactic rather than a snapshot of evaluated endpoints and orders. That qualification is appropriate, but it does not make a rule that is actually an option into a meaningful additional expansion variable. The error arises when syntactic metadata is used operationally to decide whether numerical application is possible.[34]

6.4 A narrowly tested repair

The candidate distinguishes literal option keys from key expressions that need preparation:

```
nativeLiteralOptionKeyQ[HoldComplete[_String]] := True;
nativeLiteralOptionKeyQ[HoldComplete[key_Symbol]] := OwnValues[key] === {};
nativeLiteralOptionKeyQ[_] := False;
nativeComputedArgumentQ[
  HoldComplete[(Rule | RuleDelayed)[key_, _]]] :=
  ! nativeLiteralOptionKeyQ[HoldComplete[key]];
```

The existing preparation path can then evaluate the trailing key-containing argument while retaining the source inside `HoldComplete`. This uses the current architecture instead of releasing the whole request prematurely.

In a temporary standalone containing only this replacement, the computed backend witness selected `"Series"`; a counted source `(c++;Exp[x])` was evaluated once; the `Analytic` alias yielded variable `x`; and numerical application at one returned 8/3. The old source anchor occurred once. These observations show that the demonstrated failures can be repaired without eagerly evaluating this source before backend selection.[35]

They do not prove equivalence of every possible Wolfram program with arbitrary side effects. Delayed values, unusual upvalues, symbolic/string option-name conventions, nested containers, and a key expression that changes on successive evaluations require additional acceptance tests. The emitted patch is deliberately labelled a candidate for the witnessed subset, not a universal evaluator emulation.

[33] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/NativeCompatibility.wl`, computed-argument predicates, selector preparation, and `nativeSpecificationQ`, lines 55–100..

[34] AsymptoticAnalysis, pinned revision `6687962f3c85`, `docs/development/NATIVE_RESULT_CONTRACTS.md`, input/evaluation policy and native numerical application..

[35] Accompanying `evidence/native_observations.json`, observation(s) P02; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

7 N03: the specification grammar swallows unknown options

7.1 Observed acceptance

The following request has a complete ordinary expansion specification and an extra unrecognized symbolic rule:

```
s = Quiet[AsymptoticExpansion[Exp[x], {x, 0, 3},
  unknownOption -> 7, "Backend" -> "Package"]];
{Head[s], Normal[s]}
(* {GeneralizedSeries, 1 + x + x^2/2} *)
```

The public result is a valid-looking package expansion rather than a `Failure`. Because this probe used `Quiet`, it establishes acceptance without failure, not the absence of every possible message. A production caller relying on result type rather than captured messages cannot tell that this trailing rule was ineffective.[36]

The native front end needs to support rules of the form `x->x0`; therefore a blanket ban on symbolic rules is not a correct repair. But the current complement-of-options predicate is too permissive. In effect, every symbolic spelling not found in the published option list becomes a candidate variable specification. That includes a misspelling or a future unsupported option. The stricter package path's option rejection cannot help when a rule has already been removed from the option set by misclassification.[37]

7.2 A grammar-level fix

For the package path, accept the documented source and one admitted coordinate specification, followed by option containers. Once the specification has been consumed, every trailing rule should either be a recognized option or receive a structured diagnostic. Multiple-variable native specifications belong to a separate admitted native grammar; they should not be smuggled through the analytic-package grammar as arbitrary extra arguments.

For explicit native routes, the situation is broader. Native calls may legitimately have several specifications. The parser should assign roles using the chosen backend's admitted syntax and argument positions, while preserving enough held syntax to forward the call faithfully. A rule that cannot be unambiguously classified should produce an ambiguity or unsupported-form result, rather than being silently declared an expansion variable.

This is a larger change than N02's key-preparation repair. The candidate patch emitter does not implement it. The supplied desired-contract test intentionally remains failing until a policy is implemented. Making this limitation explicit is preferable to supplying a patch that rejects a few unknown symbols but still breaks legitimate successive native specifications.

[36] Accompanying `evidence/native_observations.json`, observation(s) O07; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

[37] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/NativeCompatibility.wl`, `nativeSpecificationQ`, `nativeRequestOptionKeys`, and `packagePreparedExpansion`..

7.3 Why this is more than cosmetic validation

An accepted typo is dangerous in scientific workflows even when the numerical coefficients in the returned object happen to be correct. The ignored argument may have been intended to control assumptions, a budget, a branch, or an engine policy. N03's particular token is not itself a valid branch option, so no bypass of a correctly spelled branch constraint is asserted. The engineering concern is the inability to distinguish an honored request from one whose extra rule was discarded.

This suggests a useful postcondition: every supplied non-source argument must be accounted for exactly once as a specification, a wrapper selector, or an engine option. A result should never require the caller to infer which arguments were consumed by observing whether the printed approximation looks plausible.

8 N04: preserving a native payload while densifying its view

8.1 A concrete supported family

Let $A_n(x)$ be a length-n sparse vector whose first entry is e^x and all other entries are zero:

```
a = SparseArray[{1 -> Exp[x]}, 100000];
r = Series[a, {x, 0, 3}];
s = AsymptoticExpansion[a, {x, 0, 3}, "Backend" -> "Series"];
{Head[r], ByteCount[r], Head[s["NativeResult"]],
 ByteCount[s["NativeResult"]], Head[s["Expression"]],
 Length[s["Expression"]], ByteCount[s["Expression"]], ByteCount[s]}
```

The executed result was

```
{SparseArray, 624, SparseArray, 624,
 List, 100000, 2400120, 2404576}
```

A separate direct-native control at $n = 1000$ also returned a 624-byte `SparseArray`; its explicitly requested `Normal` was a list. These are structural `ByteCount` observations on one kernel, not peak resident-memory measurements.[38]

The wrapper did preserve `NativeResult`. The defect is the additional compulsory stored view. Merely obtaining the wrapper allocates a dense length-100000 expression even when the caller is interested only in the native sparse payload. The ratio of reported wrapper bytes to native-result bytes in this observation is approximately 3853.49, but it must not be reported as a speed ratio or a general cross-platform constant.

8.2 The causal source line

After calling the selected native engine, the constructor executes

[38] Accompanying `evidence/native_observations.json`, observation(s) O08–O09; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

```
normal = Normal[result];
```

and inserts that value as `"Expression"`. This happens before the `GeneralizedSeries` object is returned and without an explicit materialization request.[39]

`Normal` is not restricted to removing a series order term. It converts special representations into more elementary forms; in particular, `Normal` of a `SparseArray` is an ordinary array. Therefore lossless retention of the original field does not imply representation-preserving construction of the wrapper as a whole.[40,41]

Proposition 8.1 (Materialization lower bound)**.** *For the family A_n, any constructor that unconditionally stores an ordinary list of all n entries has storage and construction work at least proportional to n, regardless of the fixed number of nonzero entries in the native sparse result.*

Proof. An ordinary length-n list has n element positions. Creating and retaining those positions requires $\Omega(n)$ representation space and work in the explicit-list model. The sparse input and the observed native output have a fixed number of stored nonzero values, with dimension information stored separately. Calling `Normal` forces the explicit-list representation even though the caller has not requested it. The conclusion concerns the added view; it does not assume that every internal native series algorithm has constant running time. □

This is a sharper performance result than a suggestion to profile duplicated metadata. It supplies an admitted input family, identifies the exact eager operation, separates the preserved payload from the expensive view, and establishes the asymptotic growth mechanism without running a dangerous out-of-memory experiment.

8.3 Why the old dense-export repair does not fix it

The existing C01 repair limits optional `SeriesData` allocation when exporting a custom analytic sparse series onto a rational exponent lattice. N04 involves neither an analytic-to-native conversion nor a huge exponent gap. It is explicit native delegation applied to a sparse array that native `Series` already handled efficiently. The old export guard is not on this path. Recommending the same guard again would miss the cause.[42]

Likewise, repurposing `"MaxTerms"` is not an immediate solution. The native adapter deliberately rejects package-specific resource options rather than pretending to enforce them inside a built-in. Native materialization is a different resource contract. A useful remedy is to remove the unnecessary work from ordinary construction, not to refuse a small native payload merely because its optional dense view would be large.

[39] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/NativeCompatibility.wl`, `nativeExpansion`, lines 214 onward..

[40] Wolfram Research, *Normal*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/Normal.html`.

[41] Wolfram Research, *SparseArray*, Wolfram Language documentation, accessed September 9, 2026: `https://reference.wolfram.com/language/ref/SparseArray.html`.

[42] AsymptoticAnalysis, pinned revision `6687962f3c85`, `docs/development/CODE_REVIEW_STATUS.md`, the named register entries and complete finding crosswalk..

8.4 Repair design and tradeoffs

Store the native payload and defer the `Normal` projection until the caller explicitly asks for `Normal[s]` or the expression view. Display should continue to use the native payload, as the existing native display contract already intends. Numerical application should request the needed projection deliberately, or use a representation-preserving substitution when that behavior is defined.

A minimal design is a native association without an eagerly computed `"Expression"`, together with accessor logic that computes it on demand. A private sentinel such as `Missing["NotMaterialized"]` can represent an absent cache, but it must not be returned as the mathematical expression merely because an accessor was not updated. The `Normal` upvalue, the `"Expression"` accessor, numerical application, and any export path that currently reads the field directly must all be audited together.

Lazy caching is optional. It improves repeated explicit requests but can again retain a large dense object indefinitely. A no-cache view is simpler and keeps the stored object's representation small, at the cost of repeated work when a client repeatedly asks for the same projection. A bounded cache is a third policy. This article recommends first implementing correct demand-driven behavior without caching, then measuring whether a cache is warranted.

The repair needs two independent acceptance conditions: the native payload remains exactly equivalent to the backend output, and constructing the wrapper does not call the expensive projection. A benchmark that measures only `NativeResult` will pass even on the defective baseline. A regression should inspect total stored size over the sparse family and separately test the explicit `Normal` result when materialization is requested. The supplied storage harness is bounded at 100000 entries and records both representations.

9 N05: syntactic resolution is not a successful outcome

9.1 Observed status values

The native wrapper currently labels these results computed:

```
a = AsymptoticExpansion[$Failed, {x, 0, 3}, "Backend" -> "Series"];
b = AsymptoticExpansion[Failure["SourceFailed", <||>],
  {x, 0, 3}, "Backend" -> "Series"];
{{a["NativeEvaluationStatus"], a["NativeResult"]},
 {b["NativeEvaluationStatus"], b["NativeResult"]}}
(* {{"Computed", $Failed},
    {"Computed", Failure["SourceFailed", <||>]}} *)
```

An invalid-order `Asymptotic` control retained an unevaluated native call and was labelled `"Unresolved"`. Thus the classifier does detect one important class of incomplete evaluation; it does not distinguish failure values from other expressions whose outer expansion call disappeared.[43]

[43] Accompanying `evidence/native_observations.json`, observation(s) O10–O11; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

9.2 A documented rule with insufficient diagnostic meaning

The implemented rule is

```
If[FreeQ[result, _System`Series | _System`Asymptotic],
  "Computed", "Unresolved"]
```

and the result-contract document states this syntactic definition explicitly. Consequently the sentinel examples are not a contradiction between implementation and specification. They expose a weakness of the specification and its name. `"Computed"` should not become the sole predicate used by a future fallback controller, benchmark harness, or client application to count successful expansions.[44,45]

There are further reasons to avoid overloading one Boolean. A list can contain successful entries and a failure; an inactive expansion can be intentionally retained; a nonfinite expression can be mathematically intentional in some contexts; and an unevaluated `Series` inside user-held data need not mean that the outer native engine failed. These are design cases for testing, not additional current-kernel reproductions claimed by this audit.

9.3 A conservative additive outcome model

Retain separate diagnostic flags:

```
<|"TopLevelFailure" -> ...,
  "ContainsFailureMarker" -> ...,
  "ContainsUnresolvedExpansion" -> ...,
  "ContainsNonfiniteExpression" -> ...,
  "EvaluationOutcome" -> "Failed" | "ContainsFailure" |
                         "Unresolved" | "Returned",
  "AnalyticValidity" -> Missing["NotEstablished"]|>
```

The last field must remain independent. Returning a finite-looking value is not an analytic proof, and improving failure diagnostics must not weaken the existing missing-contract policy for Native results.

The supplied `NativeOutcomePrototype.wl` implements held syntactic inspection as an additive prototype. It does not change upstream objects and does not label an arbitrary returned expression mathematically correct. It was not executed natively during this audit. Integrating a status redesign also requires compatibility decisions for clients that already use `"NativeEvaluationStatus"`. An additive field or versioned migration is preferable to silently changing the meaning of an existing field in a patch release.

[44] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/NativeCompatibility.wl`, `NativeEvaluationStatus` field..

[45] AsymptoticAnalysis, pinned revision `6687962f3c85`, `docs/development/NATIVE_RESULT_CONTRACTS.md`, native field table and unresolved-call warning..

10 D01-local: contradictory automatic-routing documentation

The README introduction correctly says that automatic routing for native specifications, native options, and selected representation failures is implemented. Its later development-status paragraph still says that both public entry points use the package engines under `"Backend"->Automatic`, treats explicit `"Package"` as selecting that same path, and points to automatic fallback as remaining implementation work. The current dispatcher and successful complex-input automatic control contradict that later simplification.[46,47,48]

A replacement should describe the actual split: successful analytic package requests retain their contract; certain native forms/options are delegated before analytic admission; selected representation failures can fall back; explicit branch/direction/resource constraints and protected sources keep the package path; complete native input coverage remains an open target. This is narrower than promising that automatic mode always tries every backend.

The improvement is not to restate the whole guide in several places. Use a short canonical policy statement and link the detailed result contract. Then associate its two simplest executable examples with documentation checks: a regular real expansion that remains a package result and an automatic complex formal expansion that becomes Native. Ordinary link checking cannot detect a paragraph whose links are valid but whose policy is stale.

11 A contract-preserving request and result design

11.1 Resolve a request once, without erasing its program

The concrete failures suggest an incremental request representation rather than a new general-purpose algebra framework. Preserve two records: the original held arguments, and a resolved request whose argument roles and option provenance have been decided. The original is useful for debugging; the resolved record is what execution and operational metadata must share.

A proposed internal record is

```
<|"OriginalArguments" -> HoldComplete[...],
  "HeldSource" -> HoldComplete[...],
  "Specifications" -> {...},
  "ExplicitOptions" -> {...},
  "ResolvedBackend" -> "Series",
  "BackendOrigin" -> "Explicit" | "Default" | "AutomaticPolicy",
  "UnconsumedArguments" -> {},
  "NativeOrderConvention" -> ...|>
```

This is a design proposal, not a currently exported association. Its value is that the same role assignment drives forwarding, variable metadata, diagnostics, and outcome interpretation. It

[46] AsymptoticAnalysis, pinned revision `6687962f3c85`, `README.md`, introduction and Development status paragraph..

[47] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/NativeCompatibility.wl`, `automaticExpansion` and `automaticPreparedExpansion`..

[48] Accompanying `evidence/native_observations.json`, observation(s) O11; selected native Wolfram 15.0.0 Linux evaluations, not an upstream acceptance-suite log.

prevents a native engine from using one effective option list while the wrapper uses an earlier syntactic guess to infer variables.

Design principle 11.1 (Single accounting of arguments)**.** *Before an engine is invoked, each supplied trailing argument must be assigned exactly one admitted role, and any unassigned argument must produce a diagnostic. Evaluated option keys must not be used in execution while unevaluated aliases are independently reclassified as variables in metadata.*

11.2 A limited equivalence law for options

For deterministic, side-effect-free key aliases and equivalent admitted option containers, the wrapper should satisfy a useful metamorphic law: replacing a literal key by a local symbol with that same value should not change the selected engine, the approximation, the operational variable metadata, or the number of source evaluations. N02 supplies counterexamples on the baseline and a successful repaired subset.

This law must be stated with its restrictions. Wolfram Language is an evaluator, not a static JSON parser. Arbitrary side-effecting definitions can make two syntactically similar programs operationally different, and the native result-contract document already disclaims preserving every direct-call side-effect ordering. The correct response is a bounded supported equivalence policy with tests, not an unsupported promise to emulate all evaluation semantics.

Option-name normalization should also be deliberate. The ordinary `OptionValue` convention treats symbolic and string names interchangeably and ignores contexts when comparing option names. A held wrapper may choose a stricter accepted grammar, but it should document that choice and reject unsupported spellings consistently. The computed-key patch only repairs the demonstrated preparation cases; it is not a complete implementation of all `OptionValue` conventions.

11.3 Preserve native results before enriching them

A Native result should first be captured in a representation-preserving form. Optional projections, previews, and summaries can then be built on demand. This ordering avoids N04 and gives N05 a clear inspection target. It also separates three questions that are easy to conflate: what did the backend return, was that return operationally successful, and what analytic theorem, if any, is justified?

The existing missing analytic remainder and exactness fields are appropriate defaults. No proposed dispatch or storage repair should replace them with inferred guarantees based on the absence of a native head, the presence of real coefficients, or a visually plausible `Normal` expression. A future analytic promotion path needs its own admitted hypotheses and evidence; that broader project is already tracked in the repository.

11.4 Why an unconditional second backend is not the immediate fix

The register already identifies broader automatic coverage and second-backend search as unfinished work. Simply trying another backend on every non-result does not repair ignored defaults, key-role confusion, or eager materialization. It can also duplicate side effects or discard an explicitly selected policy.

Before adding retries, the wrapper needs a resolved backend policy, a stable already-evaluated or held replay boundary, and a usable outcome classification. The new findings supply concrete prerequisites for the existing coverage objective. The incremental recommendation is to make retries consume a validated resolved request and an explicit failure category; it is not to add another unstructured `Quiet[Check[...]]` around the entire public call.

12 Focused development and validation plan

12.1 Repair sequence and acceptance criteria

First repair N01 and the witnessed N02 preparation cases. They are small enough to review independently, and their tests should preserve existing analytic assumptions and native holding behavior. Integrate them through the canonical modular source, then regenerate the standalone. Editing only the generated root file would be overwritten by the repository's existing build process.[49]

Next address N04 as a result-construction change. Its acceptance criterion is representation growth, not only mathematical equality. For a fixed nonzero support and increasing array dimension, the stored wrapper must not scale like a dense list before explicit materialization. Keep explicit `Normal` compatibility tests and verify that display and numeric application use the new accessor policy.

Then implement the package/native grammar split needed for N03 and the additive outcome fields needed for N05. These are policy-bearing changes and should not be hidden inside the two minimal patches. The documentation correction should accompany the final selected semantics, not anticipate an unimplemented policy.

12.2 A targeted metamorphic matrix

A compact but effective matrix varies one request dimension at a time. Backend selection should cover literal explicit values, canonical defaults, overrides of changed defaults, and delayed values. Keys should cover literals and side-effect-free local aliases. Containers should cover admitted nested option lists and sequences. Sources should include a pure counted expression, a regular real scalar, a complex scalar delegated natively, and a sparse array. Results should include an ordinary series, a returned failure marker, and an unresolved native call.

The decisive comparisons are not all coefficient comparisons. For N01, compare selected backend and order convention. For N02, compare variable metadata and numerical application as well as the native expression. For N03, require a structured failure on an unconsumed argument. For N04, measure stored representation growth before accessing `Normal`. For N05, compare outcome flags while retaining missing analytic validity.

This matrix is more specific than another generic request for additional tests. It targets the exact intersections missed by a list of ordinary function examples. The supplied `DesiredContracts.wlt` contains fourteen desired-contract tests and controls. Some intentionally fail on the pinned baseline, and some remain failing with the two-fix candidate because that candidate does not implement the other proposals. The complete file was not run as a suite during this audit.

[49] AsymptoticAnalysis, pinned revision `6687962f3c85`, `src/Kernel/README.md`, editing and standalone-generation instructions..

12.3 Performance experiments that would be informative

The supplied native storage harness records native bytes, stored native bytes, expression bytes, total wrapper bytes, output head, and equality of the native payload. It also provides timing fields for a local run, but this article reports no timing results from that complete harness. Its bounded dimensions are 1000, 10000, and 100000; there is no need to risk a huge allocation to demonstrate linear densification.

For a future controlled timing comparison, load the package before timing, separate first-call and repeated-call measurements, and report that the wrapper may benefit from a cache warmed by the direct native control. Reverse execution order or use fresh processes when estimating timing overhead. Structural storage and runtime should remain separate reported quantities. Exact `ByteCount` totals may change by kernel version, so regression tests should use growth constraints or generous relative bounds rather than one magic byte number.

The existing register already proposes coefficient-work and provenance profiling. N04 adds a different concrete benchmark family at the native boundary. It should not be taken as evidence that the custom sparse polynomial multiplication or the inverse recurrence has the same memory behavior.

12.4 Future work with a bounded scope

The most useful next development project is a native adapter whose public semantics can be audited independently of the mathematical engines. Its scope is finite: resolve supported request forms once, preserve the selected engine policy, account for every argument, retain native representations without implicit conversion, classify returned outcomes conservatively, and expose on-demand views with explicit cost boundaries.

This does not replace the repository's existing mathematical roadmap. Complex-sector analysis, wider transseries closure, uniform parameter asymptotics, integration of germs, and stronger certificates already have entries. Recommending them again would not add a new decision. The incremental contribution here is a prerequisite layer with measurable contracts, allowing those future engines to be added without recreating the same wrapper defects.

13 Artifacts, reproducibility, and limitations

13.1 Delivered artifacts

File	Purpose and execution status
`article/audit.tex`, `article/audit.pdf`	Self-contained article source and rendered PDF.
`evidence/native_observations.json`	Thirteen grouped native observations and two independently exercised patch mechanisms, transcribed from successful responses. These are not fifteen uniform test-suite cases.
`evidence/novelty_ledger.csv`	Finding-level relationship to the existing register and evidence IDs.
`evidence/validation_scope.json`	Explicit distinction between executed checks and supplied, unrun complete programs.

File	Purpose and execution status
`code/patch_native_dispatch.py`	Fail-closed source-patch emitter for N01 and the demonstrated N02 key preparation; writes a new file only.
`code/test_audit_tools.py`	Eight synthetic emitter tests and four independent mathematical/storage checks; twelve methods passed locally.
`code/DesiredContracts.wlt`	Fourteen desired-contract specifications and controls; complete native file not executed in this audit.
`code/RunAudit.wl`	Local runner for those specifications, not the full upstream test suite; supplied but not executed here.
`code/BenchmarkNativeStorage.wl`	Bounded local measurement harness; the separate 100000-element observation was executed, not this full harness.
`code/NativeOutcomePrototype.wl`	Additive held outcome inspector; not an integrated repair and not natively executed here.

The patch emitter's tests verify unique anchors, refusal on missing or duplicated anchors, refusal to patch an already changed fixture, selection of either repair or both, and preservation of surrounding fixture text. They do not execute Wolfram Language. The independent mathematical checks confirm the displayed coefficient/frontier relations and storage-ratio arithmetic; they do not validate the package implementation.

13.2 Reproduction procedure

Load the pinned standalone, not a later moving revision:

```
Get[URLDownload[
 "https://raw.githubusercontent.com/VladimirReshetnikov/Asymptotic/6687962
    f3c858a4f93623cfc496f33e35c6763d4/AsymptoticAnalysis.wl"]];
```

The evidence file supplies the individual probe bodies. For local candidate staging, save a copy of the pinned modular `NativeCompatibility.wl` or standalone and run:

```
python code/patch_native_dispatch.py NativeCompatibility.wl candidate.wl --fix both
python -m unittest discover -s code -p test_audit_tools.py -v
```

The emitter refuses to overwrite its source or any existing output. Its exact anchors help prevent accidental application to an unrelated revision; they are not a proof that all surrounding source is unchanged. Review the generated diff against the pinned revision.

For a standalone candidate, a fresh local Wolfram process can load it and evaluate the desired-contract file. A modular candidate must be reviewed and staged into the corresponding checkout before the normal modular entry point is loaded; a companion module is not independently loadable. The root README gives both direct `TestReport` and command-line forms. The resulting failures need interpretation by finding: N03–N05 are deliberately outside the supplied two-fix patch.

No checksum files are included. The immutable repository revision, source anchors, code, and explicit observation records identify the subject and the tested mechanisms without presenting a checksum manifest as an acceptance claim.

13.3 What remains uncertain

The native observations are version- and platform-specific. A later source revision may already contain a repair, and a later kernel may change native representation details. The complete upstream suite was not run, so this article does not establish absence of regressions elsewhere. The emitted combined patch has synthetic emitter tests, but its two mechanisms were natively exercised independently rather than together as a full package.

No new mathematical counterexample is asserted for the selected real-domain, exponential-phase, or certificate code inspected here. Existing unresolved mathematical obligations remain in the upstream register and are not declared closed by this review. Conversely, a service failure during an exploratory call is not evidence that a mathematical feature is unavailable. These limits are part of the interpretation of the findings, not a substitute for the successful concrete reproductions above.

14 Conclusion

AsymptoticAnalysis has a defensible technical role alongside Wolfram Language's built-ins: preserve and operate on admitted real asymptotic expansions with richer remainder, branch, and refinement information, while allowing native results to remain native where those analytic promises have not been established. The positive irrational inverse example demonstrates a practical advantage without overstating the limitations of the native system.

The new failures show that this architecture also requires careful engineering outside the coefficient algebra. A configured engine policy must actually be read. Option aliases must not become variables. Unknown arguments must be accounted for. A compact native value must not trigger an unsolicited dense projection. A failure marker must not become indistinguishable from a useful returned expansion merely because a native head disappeared.

The first two repairs have narrowly scoped native evidence. The remaining proposals are specified with concrete acceptance conditions rather than presented as completed implementation. Addressing this boundary will make the existing mathematical capabilities more predictable and will give the repository's already-planned extensions a more reliable interface through which to reach users.

A Source inventory

The repository references throughout the article are pinned to `6687962f3c85`; the complete revision is given at the beginning. The following source groups provide the main evidentiary trail.

1. **Repository identity and architecture.** Repository README; kernel source map; public declarations and central implementation.
2. **Novelty baseline.** code-review index; wave 1; wave 2; maintained implementation register.
3. **Native contract and new findings.** NativeCompatibility.wl; native result-contract document.
4. **Selected mathematical boundaries.** SpecialFunctionRealDomain.wl; ExponentialForward.wl; InverseCertificates.wl.
5. **Official native references.** Wolfram Language documentation for Series, Asymptotic, SeriesData, InverseSeries, ComposeSeries, and AsymptoticSolve.
6. **Native adjacent solvers.** Official references for AsymptoticIntegrate, AsymptoticSum, AsymptoticDSolveValue, and AsymptoticRSolveValue.
7. **Evaluation and representation.** Official references for SetOptions, OptionValue, Normal, SparseArray, and $ScriptCommandLine.
8. **New execution evidence.** The accompanying observation JSON, novelty ledger, validation-scope JSON, and local Python-test output. These are audit artifacts, not upstream acceptance records.